BACSE202 · Database Management Systems · Fall 2026–27 · VIT Chennai

DA2: SQL & PL/SQL Explained

Satellite Imagery Knowledge Graph (GraphRAG for Earth Observation Data)

Study Guide: Every File, Every Query, Every Result

schema.sql · sample_data.sql · plsql.sql · run_procedures.sql · queries.sql · extras.sql · Verification · Embeddings · Viva Questions

Prepared for Joshwa D Y (25BCE5176) and team · Faculty: Dr. S. Geetha

Contents

Introduction: How DA2 Was Built
- Where the data and queries came from
- The six files at a glance

File 1: schema.sql (Building the Database)
- Section 1: the clean-up block
- Section 2: the 15 CREATE TABLE statements
- Section 3: 12 indexes
- Section 4: 2 views

File 2: sample_data.sql (Filling the Database)
- How the data was generated
- What an INSERT looks like
- What it deliberately does not insert
- Row counts

File 3: plsql.sql (Creating the Programs)
- The 3 functions (each returns a value; usable inside SQL)
- The package pkg_eo
- The 4 triggers (programs that run automatically)

File 4: run_procedures.sql (Running the Programs)
- Step 1: EXEC pkg_eo.compute_changes;
- Step 2: EXEC pkg_eo.raise_alerts;
- Step 3: finalize_query for each of the 4 questions
- Step 4: basin_report('MKG') and basin_report('AMZ')
- Step 5: add_detection (one valid call, one invalid)
- Step 6: trigger demonstrations

File 5: queries.sql (Asking the Database Questions)
- Group A: joins and filtering
- Group B: aggregates
- Group C: subqueries and set operators
- Group D: the project's main question
- Group E: analytics
- Group F: statistics, JSON and CASE
- Group G: GraphRAG evidence and alerts
- Group H: transactions and integrity

File 6: extras.sql (Covering the Remaining Syllabus)
- E1: DDL with ALTER TABLE
- E2: DCL with GRANT and REVOKE
- E3: explicit cursor with all attributes
- E4: FOR UPDATE and WHERE CURRENT OF
- E5: user-defined exception
- E6: stand-alone procedure with IN/OUT parameters
- E7: statement-level trigger
- E8: cascading DELETE inside a savepoint
- E9: data dictionary

Part 7: Cross-Verification and Problems Solved
- Generator predictions vs Oracle's results

All errors met during DA2 and their fixes

Part 8: The 8-Number Embeddings ("Deforested Patches Look Alike")

What an embedding is

Part 9: DA2 Viva Questions

Introduction: How DA2 Was Built

DA2 asked us to implement the DA1 design using SQL and PL/SQL, with table creation, constraints, sample records, SQL queries and PL/SQL programs, plus a handwritten report and a demonstration. This guide explains **all six SQL files**: what each one does, every query's purpose, the SQL concept it demonstrates, and the **actual result** from your runs in Oracle 21c XE (schema JOSHWA).

Where the data and queries came from

THE SAMPLE DATA WAS GENERATED `scripts/generate_data.py` (Python) generated every sample row: satellites, locations, images, detections and so on. It uses a fixed random **seed** (2026), so it produces **exactly the same data every time**. While generating, it also calculated **what the answers should be** (36 lineage links, 4 alerts, confidences 0.6121…) and saved them in `tests/expected_results.txt`.

THE SQL FILES WERE WRITTEN BY HAND Each query was designed with **two goals at once**:

1. **Answer a realistic question** an Earth-observation analyst would ask ("which images are clear enough to use?", "where is forest disappearing fastest?").
2. **Demonstrate a specific SQL feature** the course covers: joins, GROUP BY, HAVING, subqueries, set operators, window functions, hierarchical queries, JSON, transactions.

So the 19 queries work like a **checklist of SQL skills**, each shown on our own data.

THE RESULTS WERE VERIFIED When the files ran on the project laptop, Oracle's outputs **matched the generator's independent predictions exactly** (see Part 7). That shows the database logic is correct, not just that it runs.

The six files at a glance

#	File	One-line purpose	How to run it
1	schema.sql	**build** the structure: 15 tables, rules, indexes, views	F5, once
2	sample_data.sql	**fill** the tables with sample data	F5, once
3	plsql.sql	**create the programs**: functions, package, triggers	F5, once
4	run_procedures.sql	**run** the programs and show they work	F5, **once only**
5	queries.sql	**ask questions**: 19 queries + error demos	Ctrl+Enter, one at a time
6	extras.sql	**extra concepts**: ALTER, GRANT, cursors, exceptions…	F5

The order matters, because each file depends on the one before it: you can't insert data before tables exist, and you can't run procedures before they're created.

SQL Developer reminders. F5 = Run Script (whole file; output in Script Output). **Ctrl+Enter** = run one statement (output as a grid in Query Result). `SET SERVEROUTPUT ON` makes PL/SQL's printed messages appear. Always check the connection dropdown says **eo** before running.

File 1: schema.sql (Building the Database)

Purpose: creates the **empty structure** designed in DA1: tables, columns, rules, indexes and views. No data yet.

Section 1: the clean-up block

```
BEGIN
  FOR t IN (SELECT table_name FROM user_tables WHERE table_name IN ('AUDIT_LOG','ALERT', ...))
LOOP
    EXECUTE IMMEDIATE 'DROP TABLE ' || t.table_name || ' CASCADE CONSTRAINTS PURGE';
  END LOOP;
END;
/
```

- **What it does:** if our tables already exist, it **deletes them first**.
- **Why:** so the script can be **re-run safely**. Without it, a second run fails with "table already exists".
- **How:** it looks up existing table names in `user_tables` (Oracle's own list of your tables) and runs DROP TABLE for each. `CASCADE CONSTRAINTS` also removes foreign keys pointing at the table; `PURGE` skips the recycle bin.
- **Output:** "PL/SQL procedure successfully completed."

Section 2: the 15 CREATE TABLE statements

They're created **parents first** (SATELLITE before SENSOR, IMAGE_FRAME before DETECTED_OBJECT), because a foreign key can only point to a table that already exists. Each table has:

- **columns with data types**: NUMBER, VARCHAR2(60) (text up to 60), DATE, TIMESTAMP, CLOB (big text, used for JSON);
- **NOT NULL** for required columns (from DA1's total participation);
- a **PRIMARY KEY**, **FOREIGN KEYs** (with what happens on delete), **UNIQUE** constraints (candidate keys), **CHECK** constraints (valid ranges and lists) and **DEFAULT** values.

IDENTITY COLUMNS

```
image_id NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY (START WITH 10000)
```

Oracle numbers new rows automatically from **10000**. The sample data supplies its own IDs (1–9999), so new rows never clash. That's why the PL/SQL demo detection later became **id 10000**.

What each table's main rules enforce

Table	Key rules
SATELLITE	name unique; orbit one of 4 values; status ACTIVE/DECOMMISSIONED; revisit > 0

SENSOR	(satellite, sensor_name) unique (the weak entity); type OPTICAL/SAR/THERMAL; resolution > 0
RIVER_BASIN	code and name unique; latitude −90…90; longitude −180…180; area > 0
LOCATION	name unique; basin optional (SET NULL); region type from a list of 7
IMAGE_FRAME	file path unique; (sensor, location, time) unique; cloud 0–100; sun 0–90; embedding must be JSON
OBJECT_TYPE	type name unique
DETECTION_MODEL	(name, version) unique
DETECTED_OBJECT	confidence 0–1; area > 0; bbox min < max; JSON checks; deleted with its image (CASCADE)
OBJECT_CHANGE	(prev, curr) unique; prev ≠ curr; change type from a list; days > 0
SPECTRAL_INDEX	(image, index name) unique; min ≤ mean ≤ max, all between −1 and 1
APP_USER	email unique and email-shaped; role from a list
QUERY_LOG	confidence 0–1; weights must be JSON
QUERY_EVIDENCE	(query, object) unique; all scores 0–1; rank > 0
ALERT	one alert per change (UNIQUE); severity and status from lists; resolved_at only when RESOLVED
AUDIT_LOG	operation must be INSERT/UPDATE/DELETE

What happens when a parent row is deleted

Option	Effect	Used for
ON DELETE CASCADE	children are deleted too	sensor→satellite, object→image, index→image, change/evidence→object, alert→change
ON DELETE SET NULL	children stay; the link is emptied	location→basin, query→user, alert→assigned user
(nothing) = restrict	the delete is refused (ORA-02292)	image→sensor: don't lose image history silently

Output: "Table SATELLITE created." … "Table AUDIT_LOG created." (15 times)

Section 3: 12 indexes

```
CREATE INDEX ix_obj_image ON detected_object(image_id);
```

INDEX Like the index at the back of a book: it lets Oracle **find rows without reading the whole table**. We indexed every foreign key because (1) joins (object → image) become faster, and (2) without it, deleting a parent row can **lock the whole child table**.

Output: "Index IX_SENSOR_SAT created." … (12 times)

Section 4: 2 views

A **view** is a **saved query you can use like a table**. It stores no data itself.

- **v_image_summary** joins image + sensor + satellite + location + basin (5 tables), so each image row shows all the useful names. Many queries and the app use it instead of repeating the 5-table join.
- **v_forest_change** joins each change to **both** objects, **both** images, the location and the basin. Each row is a complete description of one change: where, when, how much, how fast, and both images' confidence and cloud cover.

Output: "View V_IMAGE_SUMMARY created.", "View V_FOREST_CHANGE created."

PROBLEM WE HIT On the first run, the views failed with **ORA-01031: insufficient privileges**, because JOSHWA had no CREATE VIEW permission. A user can't grant itself privileges, so we fixed it in SQL*Plus as the administrator:

```
CONNECT / AS SYSDBA
ALTER SESSION SET CONTAINER = XEPDB1;
GRANT CREATE VIEW, CREATE PROCEDURE, CREATE TRIGGER, CREATE SEQUENCE TO JOSHWA;
```

Then re-ran schema.sql (safe, because of the clean-up block).

File 2: sample_data.sql (Filling the Database)

Purpose: inserts the **sample records**: about **490 INSERT statements**.

How the data was generated

`generate_data.py` did the following (none of it was typed by hand):

1. defined 5 satellites, 6 sensors, 5 basins, 12 locations, 8 object types, 3 models and 4 users;
2. created **5 dates** (10 Apr, 18 Jun, 22 Jul, 27 Aug, 29 Sep 2026) × **12 locations** = **60 images**;
3. gave each forest location a **behaviour**: **rapid** loss (Manaus, Kisangani, Tonle Sap, Bhadrachalam), **slow** loss (Rondonia, Kratie, Rajahmundry), **stable** (Mbandaka, Belgrade);
4. made the tree-patch **area shrink** image by image according to that behaviour;
5. added other objects per place (buildings and ships at ports, water near rivers…), giving **247 objects**;
6. added **NDVI and NDWI** per image (NDVI falls as trees are lost), giving **120 rows**;
7. made the 8-number **embeddings** so that deforested patches look alike (explained in Part 8);
8. pre-computed **4 sample questions** with their top-5 evidence each, giving **20 evidence rows**;
9. wrote everything as INSERT statements, **parents first**, plus CSV files for Neo4j (DA3).

What an INSERT looks like

```
INSERT INTO satellite (satellite_id, satellite_name, operator_name, launch_date, orbit_type,
sat_status, revisit_days)
VALUES (1, 'Sentinel-2A', 'ESA/Copernicus', DATE '2015-06-23', 'SUN-SYNCHRONOUS', 'ACTIVE', 10);
```

`DATE '2015-06-23'` and `TIMESTAMP '2026-09-29 05:30:00'` are how you write dates in Oracle SQL.

What it deliberately does not insert

OBJECT_CHANGE, ALERT, AUDIT_LOG and the answer confidences are left empty, because the PL/SQL programs (file 4) **produce** them.

Row counts

Table	Rows	Table	Rows
SATELLITE	5	IMAGE_FRAME	60
SENSOR	6	DETECTED_OBJECT	247
RIVER_BASIN	5	SPECTRAL_INDEX	120
LOCATION	12	APP_USER	4

OBJECT_TYPE	8	QUERY_LOG	4
DETECTION_MODEL	3	QUERY_EVIDENCE	20

Output: many "1 row inserted." lines, ending with **"Commit complete."** COMMIT saves everything permanently. **Check:** `SELECT COUNT(*) FROM detected_object;` → **247**.

File 3: plsql.sql (Creating the Programs)

Purpose: creates the **programs stored inside the database**. Nothing runs yet; they're only defined here. File 4 runs them.

The 3 functions (each returns a value; usable inside SQL)

fn_haversine_km(lat1, lon1, lat2, lon2)

Purpose: the **distance in km** between two points on Earth. **Why it's needed:** for "near a river". Earth is round, so a simple Pythagoras calculation on degrees would be wrong. The haversine formula uses Earth's radius (6371 km):

$$a = \sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2) \qquad d = 2 \cdot 6371 \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{(1-a)})$$

φ = latitude, λ = longitude, both in radians. **Example:** Kisangani to the Congo river point = **38 km**.

fn_classify_change(pct)

Turns a percentage into a label: ≥ 10 → DEFORESTATION; ≤ −10 → REGROWTH; ≥ 3 → GRADUAL_LOSS; otherwise STABLE.

fn_forest_loss_pct(location, from_date, to_date)

The % of forest lost at a location between two dates. It finds the tree area in the **first** and **last** image of the window (using `MAX(area) KEEP (DENSE_RANK FIRST/LAST ORDER BY capture_time)`), then calculates (first − last) / first × 100.

The package pkg_eo

PACKAGE A named box grouping related programs. It has a **specification** (the list of what's inside, like a menu) and a **body** (the actual code). Benefits: organisation, shared definitions, and it's loaded as one unit.

```
CREATE OR REPLACE PACKAGE pkg_eo AS
  e_no_evidence EXCEPTION;  PRAGMA EXCEPTION_INIT(e_no_evidence, -20001);
  PROCEDURE compute_changes;
  PROCEDURE raise_alerts (p_min_loss NUMBER DEFAULT 10, p_min_conf NUMBER DEFAULT 0.7,
p_max_cloud NUMBER DEFAULT 40);
  FUNCTION  fn_query_confidence (p_query_id NUMBER) RETURN NUMBER;
  PROCEDURE finalize_query (p_query_id NUMBER);
  PROCEDURE add_detection (p_image_id NUMBER, p_type_name VARCHAR2, p_model_id NUMBER, p_conf
NUMBER,
                           p_lat NUMBER, p_lon NUMBER, p_area NUMBER, p_new_id OUT NUMBER);
  PROCEDURE basin_report (p_basin_code VARCHAR2);
END pkg_eo;
```

Member	Type	Purpose	PL/SQL features shown

compute_changes	procedure	**builds the EVOLVED_FROM lineage**: pairs each tree patch with the previous one at the same place and stores the change	explicit cursor, LAG() window function, loop, MERGE
raise_alerts	procedure	creates alerts **only for trustworthy** big losses	DEFAULT parameters, cursor loop, NOT EXISTS
fn_query_confidence	function	calculates an answer's **trust score**	SELECT INTO, aggregates, NULLIF
finalize_query	procedure	saves that score and a sentence on the question	RAISE_APPLICATION_ERROR, CASE
add_detection	procedure	safely inserts a new detected object, undoing itself on error	SAVEPOINT / ROLLBACK TO, OUT parameter, RETURNING INTO, exception handlers
basin_report	procedure	prints forest loss for every location in a basin	parameterised cursor, BOOLEAN flag, RPAD

The 4 triggers (programs that run automatically)

Trigger	When it fires	What it does
trg_obj_audit	after any insert/update/delete on DETECTED_OBJECT, **once per row**	writes the old and new values into AUDIT_LOG (uses INSERTING/UPDATING and :OLD/:NEW)
trg_img_validate	before an image's capture time is inserted or changed	**blocks future dates**. A CHECK constraint can't do this, because it can't use "today"
trg_alert_resolve	before an alert's status changes	**automatically fills resolved_at** when the status becomes RESOLVED; clears it otherwise
trg_change_validate	before a lineage link is inserted or updated	both objects must be at the **same location**, and the earlier one must be **older**

Output: "Function FN_HAVERSINE_KM compiled" (×3 functions), "Package PKG_EO compiled", "Package Body PKG_EO compiled", "Trigger … compiled" (×4), "No errors."

There was a harmless "pkg_eo – Unknown Command" line, caused by a `SHOW ERRORS PACKAGE BODY` line SQL Developer didn't like. We changed it to `SHOW ERRORS;`.

File 4: run_procedures.sql (Running the Programs)

Purpose: runs the PL/SQL from file 3 and **shows it works**. It also **creates** the lineage, alerts, confidences and audit rows.

RUN IT ONCE ONLY Each run inserts another demo detection (248 → 249 → 250…). If that happens: `DELETE FROM detected_object WHERE object_id > 10000; COMMIT;`. To re-show its result tables, run only the final SELECTs with Ctrl+Enter.

Step 1: EXEC pkg_eo.compute_changes;

1. finds every **Tree Coverage** patch with its image date;
2. uses **LAG()** to fetch the **previous** patch at the **same location**;
3. calculates: **change %** = (previous area − current area) / previous area × 100; **days** between the images; **rate** = change % / days × 30 (% per month); **type** = fn_classify_change(change %);
4. stores the result with **MERGE**: update the link if it exists, insert it if not. So re-running never creates duplicates.

Result: `compute_changes: 36 lineage links created/updated.`
Meaning: 9 forest locations × 4 consecutive image pairs = **36**. The 3 non-forest places have no tree patches.

Step 2: EXEC pkg_eo.raise_alerts;

Raises an alert **only if all of these hold**: loss ≥ **10%**; **both** detections' confidence ≥ **0.7**; **both** images' cloud ≤ **40%**; no alert exists yet for that change. Severity: ≥ 25% CRITICAL, ≥ 15% HIGH, otherwise MEDIUM.

Result: `raise_alerts: 4 new alerts.`
Meaning: 10 changes are DEFORESTATION, but **only 4 had confident, clear-sky evidence**. The other 6 were **deliberately not alerted**, which avoids false alarms from bad images (novelty 3).

Step 3: finalize_query for each of the 4 questions

```
confidence = Σ (retrieval_score × detection_confidence × (1 − cloud/100)) ÷ Σ retrieval_score
```

```
finalize_query(1): Answer based on 5 evidence objects. Overall confidence 61.2% (moderate - ...)
finalize_query(2): ... 66.6% ...
finalize_query(3): ... 61.8% ...
finalize_query(4): ... 60.1% ...
```

Meaning: all four are "moderate", because some evidence came from cloudy images. The exact values (**0.6121, 0.6655, 0.618, 0.6006**) **matched the generator's prediction to**

4 decimals.

Step 4: basin_report('MKG') and basin_report('AMZ')

```
=== Forest loss report, basin MKG (Apr-Sep 2026) ===
Kratie Riverbank            7.74 % loss
Tonle Sap Fringe            38.49 % loss
=== Forest loss report, basin AMZ (Apr-Sep 2026) ===
Manaus Forest Edge          38.72 % loss
Rondonia Frontier           8.13 % loss
```

Meaning: a cursor listed each basin's locations and called fn_forest_loss_pct. In each basin, one place is deforesting **rapidly** (about 38%) and one **slowly** (about 8%).

Step 5: add_detection (one valid call, one invalid)

```
Inserted detection id = 10000
Expected error caught: ORA-20020: Unknown object type: Spaceship
```

Meaning: the valid Building (confidence 0.91, area 850 m²) was inserted and its new id returned through the **OUT parameter**, so the count went **247 → 248**. The "Spaceship" was **rejected**, and the package's SAVEPOINT/ROLLBACK undid the attempt.

Step 6: trigger demonstrations

6a. Inserting an image dated 2030 gave `ORA-20010: capture_time cannot be in the future:` trg_img_validate **blocked it**.

6b. Two updates: alert 10000 set to **RESOLVED** (assigned to user 1), and object 1's confidence set to **0.99**. The result tables at the end:

ALERT_ID	SEVERITY	STATUS	ASSIGNED	RESOLVED_AT
10000	HIGH	RESOLVED	1	27-OCT-26 12.53... ← **filled automatically**
10001	MEDIUM	OPEN		
10002	MEDIUM	OPEN		
10003	HIGH	OPEN		

AUDIT_ID	TABLE	OPERATION	RECORD	OLD	NEW
1	DETECTED_OBJECT	INSERT	10000		conf=.91, area=850
2	DETECTED_OBJECT	UPDATE	1	conf=.837, area=5949857.19	conf=.99, area=5949857.19

trg_alert_resolve set the resolved time by itself; trg_obj_audit recorded both changes by itself. The query-log table shows the 4 confidences with their sentences.

Script Output prints tables as plain text, so wide columns (old_value, response_text) wrap across lines. For neat screenshots, run just those SELECTs with Ctrl+Enter to get a grid.

File 5: queries.sql (Asking the Database Questions)

Purpose: 19 queries plus 3 error demonstrations. Each one answers a **real analyst question** and demonstrates a **specific SQL feature**. They're grouped A–H by technique. Run them **one at a time with Ctrl+Enter** so each result appears as a grid you can screenshot.

Group	Queries	SQL skill shown
A	Q1–Q3	joins, views, filtering, date ranges
B	Q4–Q6	GROUP BY, aggregates, HAVING, multi-hop joins
C	Q7–Q9	subqueries (non-correlated, correlated), MINUS
D	Q10–Q11	the project's target question in SQL, plus trust
E	Q12–Q13	window functions, hierarchical (recursive) query
F	Q14–Q16	statistics (CORR), JSON, CASE
G	Q17–Q18	GraphRAG evidence and the alert dashboard
H	Q19–Q20	transactions and integrity violations

Group A: joins and filtering

Q1: images with satellite, sensor and location

```
SELECT image_id, capture_time, satellite_name, sensor_name, location_name, cloud_cover
FROM v_image_summary ORDER BY capture_time, image_id FETCH FIRST 10 ROWS ONLY;
```

Question: "Show me the images with readable names, not just ID numbers." **Concept:** a **view** hiding a 5-table join; **FETCH FIRST n ROWS ONLY** limits the output. **Result: 10 rows**, all from **10-Apr-26** (the earliest date), e.g. image 1, Manaus, Sentinel-2B, MSI, 31.4% cloud.

Q2: locations not in any river basin

```
SELECT l.location_name, l.country, l.region_type
FROM location l LEFT JOIN river_basin b ON b.basin_id = l.basin_id
WHERE b.basin_id IS NULL;
```

Concept: the **anti-join** pattern. A LEFT JOIN keeps *every* location; where no basin matches, the basin columns are NULL, and the WHERE keeps only those. **Result: 3 rows**: **Chennai Coast, Rotterdam Port, Dubai Marina**, the coastal/urban places (that's why basin_id was optional in DA1).

Q3: clear-sky images in Q3 2026

Filters July–September with `capture_time >= '2026-07-01' AND < '2026-10-01'` and `cloud_cover < 20`. **Why:** cloudy images are useless for optical analysis. **Concept:** a

TIMESTAMP range (half-open: ≥ start, < end, so no day is missed or double counted). **Result: 12 rows**; clearest is **image 23, Tonle Sap, 3.7%**.

Group B: aggregates

Q4: images and average cloud per satellite

```
SELECT s.satellite_name, COUNT(*) AS images, ROUND(AVG(i.cloud_cover),1) AS avg_cloud
FROM image_frame i JOIN sensor sn ON sn.sensor_id = i.sensor_id
JOIN satellite s ON s.satellite_id = sn.satellite_id
GROUP BY s.satellite_name ORDER BY images DESC;
```

Concept: GROUP BY makes one row per satellite; COUNT and AVG summarise each group. **Result: 4 satellites × 15 images**; average cloud 28 / 32.5 / 26.5 / 27%.

VIVA POINT We have **5** satellites, but only 4 rows appear. **Sentinel-1A has no images**, and an **inner join** drops rows with no match. To show it with 0, use a LEFT JOIN from SATELLITE, which is exactly what E3 does.

Q5: detections per object type (HAVING)

```
SELECT t.type_name, t.category, COUNT(*) detections, ROUND(AVG(o.confidence_score),3) avg_conf
FROM detected_object o JOIN object_type t ON t.object_type_id = o.object_type_id
GROUP BY t.type_name, t.category HAVING COUNT(*) > 5 ORDER BY detections DESC;
```

Type	Detections	Avg confidence
Building	56	0.816
Tree Coverage	45	0.793
Water Body	40	0.828
Vessel	35	0.812
Road	25	0.825
Cropland	25	0.827
Bare Soil	20	0.862

Concept: WHERE filters rows before grouping; HAVING filters groups after. There are 8 types but only 7 rows: **Wildfire Scar** has too few detections, so HAVING removes its group.

Q6: objects per river basin (4-hop join)

Basin → location → image → object, all with LEFT JOINs and COUNT(o.object_id). **Result: Godavari 45, Mekong 35, Amazon 33, Congo 30, Danube 20**. The objects at the 3 non-basin locations aren't counted, because the query starts from RIVER_BASIN.

Group C: subqueries and set operators

Q7: images cloudier than average (non-correlated)

```
WHERE cloud_cover > (SELECT AVG(cloud_cover) FROM image_frame)
```

The inner query runs **once**, gives one number (the overall average), and every row is compared with it. **Result:** top **8** shown; cloudiest is **image 18, Mbandaka, 53.3%**.

Q8: latest tree area per location (correlated)

```
AND i.capture_time = (SELECT MAX(i2.capture_time) FROM image_frame i2 JOIN detected_object o2 ...
                      WHERE i2.location_id = i.location_id ...)
```

The inner query refers to the outer row's location (`i.location_id`), so it's re-evaluated **per row**: "the latest date *for this location*". **Result: 9 rows** (one per forest location); **Kisangani** has the smallest remaining forest, **2,611,203 m² (≈ 2.61 km²)**.

Q9: deforested locations with no alert (MINUS)

```
SELECT DISTINCT location_name FROM v_forest_change WHERE change_type = 'DEFORESTATION'
MINUS
SELECT DISTINCT fc.location_name FROM v_forest_change fc JOIN alert a ON a.change_id =
fc.change_id;
```

Concept: MINUS = rows in the first result that are not in the second (set difference). **Result: Manaus, Bhadrachalam**. They had real deforestation, but their evidence was too cloudy or uncertain, so **raise_alerts deliberately skipped them**. This is direct proof of the trust-aware alert novelty.

Group D: the project's main question

Q10: "rapid deforestation near river basins in Q3"

```
SELECT fc.location_name, fc.basin_name, fc.curr_time, fc.change_pct, fc.rate_pct_per_month,
       ROUND(fn_haversine_km(l.latitude, l.longitude, b.river_ref_lat, b.river_ref_lon))
km_to_river
FROM v_forest_change fc JOIN location l ON l.location_id = fc.location_id
JOIN river_basin b ON b.basin_id = l.basin_id
WHERE fc.change_type = 'DEFORESTATION'
  AND fc.curr_time >= TIMESTAMP '2026-07-01 00:00:00' AND fc.curr_time < TIMESTAMP '2026-10-01
00:00:00'
  AND fn_haversine_km(l.latitude, l.longitude, b.river_ref_lat, b.river_ref_lon) <= 300
ORDER BY fc.rate_pct_per_month DESC;
```

Each phrase of the question maps to one condition:

Words in the question	SQL
"deforestation"	change_type = 'DEFORESTATION'
"in Q3"	curr_time between 1 Jul and 30 Sep
"near river basins"	our own PL/SQL function fn_haversine_km ≤ 300 km, used inside WHERE
"rapid"	ORDER BY rate_pct_per_month DESC (fastest first)

Result: 10 rows covering **Manaus, Kisangani, Tonle Sap and Bhadrachalam** (the 4 "rapid" places from the generator); distances **38–174 km**; fastest rate **16.15% per month**. The SQL version has no "meaning" search; the semantic (vector) version is in DA3 and the app.

Q11: the same events ranked by trust

```
ROUND(LEAST(prev_conf, curr_conf) * (1 - GREATEST(prev_cloud, curr_cloud) / 100), 3) AS
evidence_trust
```

Idea: a change is only as trustworthy as its **weakest** evidence, so take the **lower** confidence (LEAST) and the **higher** cloud (GREATEST).

WORKED EXAMPLE Confidences 0.82 and 0.79 → LEAST = 0.79. Clouds 12% and 25% → GREATEST = 25 → 1 − 0.25 = 0.75. Trust = 0.79 × 0.75 = **0.593**.

Result: best **Kisangani 29-Sep, 0.584**; worst **Manaus 27-Aug, 0.306** (cloudy). This matches Q9: Manaus is low-trust, so it got no alert.

Group E: analytics

Q12: cumulative loss and rank (window functions)

```
SUM(change_pct) OVER (PARTITION BY location_id ORDER BY curr_time) AS cumulative_loss_pct,
RANK()          OVER (PARTITION BY curr_time   ORDER BY change_pct DESC) AS rank_in_period
```

WINDOW FUNCTION Calculates across **related rows** but, unlike GROUP BY, **keeps every row**. **PARTITION BY** = which rows belong together; **ORDER BY** inside OVER = the order for running totals or ranks.

Result: 36 rows (all lineage links). **Cumulative example:** Kisangani reaches **13.87%** total loss by 22-Jul (its June and July losses added up). **Rank example:** within each date, the location with the biggest loss gets rank 1.

Q13: full lineage chains (hierarchical query)

```
SELECT LEVEL, SYS_CONNECT_BY_PATH(TO_CHAR(prev_object_id), ' -> ') || ' -> ' || curr_object_id AS
chain
FROM object_change
START WITH prev_object_id NOT IN (SELECT curr_object_id FROM object_change)
CONNECT BY PRIOR curr_object_id = prev_object_id;
```

- **START WITH**: begin at patches that are nobody's "current", i.e. the **first** image's patch.
- **CONNECT BY PRIOR**: follow each link to the next (this link's curr = next link's prev), recursively.
- **LEVEL** = step number; **SYS_CONNECT_BY_PATH** builds the path text.

Result: 36 rows (one per step), e.g. `1 → 3 → 5 → 7 → 9` (Manaus across all 5 dates) and `33 → 35 → 37 → 39 → 41`. This is the **EVOLVED_FROM** recursive relationship from DA1, traversed in SQL. Neo4j does the same with `[:EVOLVED_FROM*]`.

Group F: statistics, JSON and CASE

Q14: does NDVI agree with tree area? (CORR)

`CORR(x, y)` gives the Pearson correlation (−1 to +1). **Result: 0.442**, a moderate positive correlation: more forest goes with greener NDVI, as expected. It isn't higher because each location starts with a different forest size.

Q15: vessels by class (JSON)

```
JSON_VALUE(attributes_json, '$.vessel_class')                    -- text from inside the JSON
AVG(JSON_VALUE(attributes_json, '$.length_m' RETURNING NUMBER))   -- number from inside the JSON
```

Different object types have different details (a vessel has a class and length; a building has floors), so they're stored as flexible JSON. **Result: cargo 11 (avg 127.3 m), tanker 9 (160.7 m), passenger 8 (138.8 m), fishing 6 (101 m)**.

Q16: resolution classes (CASE)

CASE turns a number into a label (≤ 10 m High, ≤ 30 m Medium, else Coarse), and we group by that label. **Result: High 30 images (avg cloud 27.5%), Medium 30 (29.5%)**.

Group G: GraphRAG evidence and alerts

Q17: evidence behind each question

Joins 6 tables (query_log → query_evidence → object → type → image → location) and keeps the top 3 ranks. **Result: 12 rows** (4 questions × 3). Question 1's top evidence is a **Kisangani tree patch, score 0.9484**. This is the "explainable" part of GraphRAG: every answer shows its sources.

Q18: alert dashboard

```
ORDER BY CASE a.severity WHEN 'CRITICAL' THEN 1 WHEN 'HIGH' THEN 2 ELSE 3 END, fc.change_pct DESC
```

Concept: a **custom sort order** (alphabetical would put HIGH before CRITICAL incorrectly); LEFT JOIN so unassigned alerts still appear. **Result: 4 alerts**: 10000 HIGH Kisangani RESOLVED (Aarav Menon), 10003 HIGH Tonle Sap, then 10002 and 10001 MEDIUM Tonle Sap.

Group H: transactions and integrity

Q19: SAVEPOINT and ROLLBACK

```
SAVEPOINT s1;
UPDATE image_frame SET sun_elevation = sun_elevation + 1 WHERE location_id = 1;   -- 5 rows
updated
ROLLBACK TO s1;                                                                    -- Rollback
complete
```

Shows a change can be **undone before COMMIT**. The data is unchanged afterwards.

Q20: integrity violations (each should fail)

Attempt	Error	Rule that stopped it
image with cloud_cover = 140	ORA-02290	CHECK (0–100)
object in image 99999 (doesn't exist)	ORA-02291	FOREIGN KEY: parent missing
delete satellite 1	ORA-02292	FOREIGN KEY: child records exist (no cascade from sensor to image)

Why show failures? To prove the database **protects itself**: bad data is rejected by the DBMS, not just by the app.

File 6: extras.sql (Covering the Remaining Syllabus)

Purpose: files 1–5 didn't show some course topics: ALTER, GRANT/REVOKE, explicit cursor attributes, FOR UPDATE, user-defined exceptions, a stand-alone procedure, a statement-level trigger, cascading DELETE and the data dictionary. extras.sql adds them. Demo changes are **rolled back** or reversed so the main data stays clean.

E1: DDL with ALTER TABLE

```
ALTER TABLE satellite ADD (launch_mass_kg NUMBER(6));
ALTER TABLE satellite MODIFY (launch_mass_kg NUMBER(7));
ALTER TABLE satellite ADD CONSTRAINT ck_sat_mass CHECK (launch_mass_kg > 0);
ALTER TABLE satellite RENAME COLUMN launch_mass_kg TO mass_kg;
ALTER TABLE image_frame DISABLE CONSTRAINT ck_img_cloud;   ... ENABLE CONSTRAINT ck_img_cloud;
COMMENT ON TABLE object_change IS '...';
ALTER TABLE satellite DROP COLUMN mass_kg;
```

Result: satellites listed with masses **2711 / 2623 / 2300 / 1140 / 1130 kg** (Landsat 9 heaviest, Sentinel-2B lightest); CK_IMG_CLOUD status **DISABLED → ENABLED**; the table comment shown; the column finally dropped so the schema returns to normal.

E2: DCL with GRANT and REVOKE

Gives SELECT on the view and SELECT/INSERT on query_log to PUBLIC, lists them from `user_tab_privs_made`, then revokes all three. **Result: 3 privileges → 0**.

E3: explicit cursor with all attributes

```
CURSOR c_sat IS SELECT ... FROM satellite s LEFT JOIN ... GROUP BY ...;
r c_sat%ROWTYPE;
OPEN c_sat;  LOOP FETCH c_sat INTO r; EXIT WHEN c_sat%NOTFOUND; ... END LOOP;
DBMS_OUTPUT.PUT_LINE('Rows fetched (%ROWCOUNT) = ' || c_sat%ROWCOUNT);  CLOSE c_sat;
```

%ROWTYPE = a variable shaped like one row; **%NOTFOUND** = true when no row was fetched; **%ROWCOUNT** = rows fetched so far; **%ISOPEN** = is it open. **Result: 5 satellites**, including **Sentinel-1A with 0 images** (the LEFT JOIN keeps it, unlike Q4); "Rows fetched (%ROWCOUNT) = 5".

E4: FOR UPDATE and WHERE CURRENT OF

Locks the open MEDIUM alerts, and for each one with loss ≥ 13% **escalates** it to HIGH using `UPDATE alert ... WHERE CURRENT OF c_alert` (updates exactly the row the cursor is on). Then ROLLBACK. **Result: alert 10002 (13.91%) escalated**, then undone.

BUG AND FIX Version 1's cursor joined ALERT with OBJECT_CHANGE and gave **ORA-01410: invalid ROWID**, because WHERE CURRENT OF must point to a row of **one** table. **Fix:** the cursor reads ALERT only, and the change % is fetched inside the loop with `SELECT change_pct INTO v_pct FROM object_change WHERE change_id = r.change_id`.

E5: user-defined exception

```
e_too_cloudy EXCEPTION;   c_max_cloud CONSTANT NUMBER := 40;   v_cloud
image_frame.cloud_cover%TYPE;
...  IF v_cloud > c_max_cloud THEN RAISE e_too_cloudy; END IF;
EXCEPTION WHEN e_too_cloudy THEN DBMS_OUTPUT.PUT_LINE('... rejected ...');
```

We **define our own error**, **raise** it when a business rule fails, and **catch** it. **Result: image 23** (Tonle Sap, 3.7%) **accepted**; **image 18** (Mbandaka, 53.3%) **rejected: exceeds 40%**.

E6: stand-alone procedure with IN/OUT parameters

`proc_satellite_stats(p_name IN, p_images OUT, p_avg_cloud OUT, p_objects OUT)`. The caller passes a name in and receives three values out. **Result:** "**Sentinel-2A: 15 images, avg cloud 28%, 61 detected objects**"; calling it with "**Hubble**" (not in our table) → NO_DATA_FOUND converted to **ORA-20040** and caught.

E7: statement-level trigger

```
CREATE OR REPLACE TRIGGER trg_alert_stmt_audit
AFTER INSERT OR UPDATE OR DELETE ON alert          -- no FOR EACH ROW → statement level
DECLARE v_op VARCHAR2(10);
BEGIN
  IF INSERTING THEN v_op := 'INSERT'; ELSIF UPDATING THEN v_op := 'UPDATE'; ELSE v_op :=
'DELETE'; END IF;
  INSERT INTO audit_log (table_name, operation, record_id, new_value)
  VALUES ('ALERT', v_op, NULL, 'statement-level trigger: ALERT table modified');
END;
```

Result: an UPDATE touched **3 rows** but the trigger added only **1 audit row**. A row-level trigger (FOR EACH ROW, like trg_obj_audit) would add 3. That's the difference.

BUG AND FIX Version 1 computed the operation name using `CASE WHEN INSERTING …` inside the SQL INSERT. INSERTING is a PL/SQL-only predicate, so the trigger compiled **invalid** and every ALERT change failed with **ORA-04098: trigger is invalid**. **Fix:** work out v_op with a PL/SQL IF first, then insert v_op.

E8: cascading DELETE inside a savepoint

```
SAVEPOINT before_delete;
DELETE FROM image_frame WHERE image_id = 60;   -- cascades to detected_object, spectral_index
ROLLBACK TO before_delete;
```

Result: objects in image 60: **6 → 0 → 6**. Deleting one image automatically removed its 6 objects (ON DELETE CASCADE), and the rollback brought everything back.

E9: data dictionary

```
SELECT object_type, COUNT(*) FROM user_objects GROUP BY object_type ORDER BY object_type;
```

Type	Count	Explanation
FUNCTION	3	haversine, classify, forest loss
INDEX	46	12 ours + automatic ones for every PK, UNIQUE and LOB column
PACKAGE / BODY	1 / 1	pkg_eo spec and body
PROCEDURE	1	proc_satellite_stats (E6)
TABLE	15	our 15 tables
TRIGGER	5	4 from plsql.sql + E7
VIEW	2	v_image_summary, v_forest_change

Rolling back extras. Most extras undo themselves. If SQL Developer asks "Commit or Rollback?" on closing, choose **Rollback**. If you want to remove the E6 procedure and E7 trigger: `DROP TRIGGER trg_alert_stmt_audit; DROP PROCEDURE proc_satellite_stats;` (then E9 shows PROCEDURE 0, TRIGGER 4).

Part 7: Cross-Verification and Problems Solved

Generator predictions vs Oracle's results

generate_data.py applied **the same formulas** in Python on the same data and wrote what Oracle *should* output. Two independent implementations agreeing is strong evidence both are right.

Item	Python predicted	Oracle produced
lineage links	36	36 ✔
deforestation events / alerts	10 / 4	10 / 4 ✔
confidences q1–q4	0.6121, 0.6655, 0.6180, 0.6006	identical to 4 dp ✔
basin report MKG / AMZ	7.74, 38.49 / 38.72, 8.13	identical ✔
detected objects	247 (+1 demo = 248)	248 ✔

All errors met during DA2 and their fixes

Error	When	Cause	Fix
ORA-01031	schema.sql views	no CREATE VIEW privilege	GRANT as SYSDBA in XEPDB1
"Unknown Command"	plsql.sql	SHOW ERRORS PACKAGE BODY syntax	SHOW ERRORS;
count 250	run_procedures run 3×	demo insert repeated	DELETE … WHERE object_id > 10000
ORA-04098	extras E7	INSERTING used in SQL	IF in PL/SQL first
ORA-01410	extras E4	WHERE CURRENT OF on a join	single-table cursor
script "stuck"	extras	row lock held by another session	commit/rollback the other session
"could not install modules"	SQL Developer start	corrupted settings folder	rename the system26.2… folder

Part 8: The 8-Number Embeddings ("Deforested Patches Look Alike")

What an embedding is

EMBEDDING A **list of numbers describing what something "looks like"**, produced by an AI model. Things that look similar get **similar lists**, i.e. arrows pointing in a similar direction. Real systems use 512 or 768 numbers; we used **8** to keep it readable. They're stored as JSON in `embedding_json`.

We have no real AI model, so the generator **built the numbers deliberately** so they behave like real ones would. The recipe:

1. **One random "base" direction per object type** (base[Tree], base[Building]…). Every object starts near its type's base, so all buildings resemble each other.
2. **One extra "loss direction"** (loss_dir), meaning "looks like cleared forest".
3. **Tree patch** = base[Tree] + loss_dir × **6 × cumulative loss** + small noise (0.04). A patch with 0% loss sits on the plain "forest" direction; the more it has lost, the further it leans towards loss_dir.
4. **Bare soil** = base[Bare Soil] + 0.5 × loss_dir + noise, since cleared land partly looks like deforestation too.
5. **Other types** = base + noise (0.15).
6. Every vector is **normalised** to length 1, so only the direction matters.
7. An **image's** embedding = the normalised **average** of its objects' embeddings.

WHY THIS MEANS "DEFORESTED PATCHES LOOK ALIKE" Manaus (38% lost) and Kisangani (heavily lost) both lean strongly towards loss_dir, so their arrows point in nearly the **same direction** and their cosine similarity is high (close to 1). Stable Belgrade (≈0%) stays on plain base[Tree], so it's further away. A building points somewhere else entirely.

The question vector. For question 1 ("rapid deforestation…") the generator used **base[Tree] + 2.4 × loss_dir**, i.e. "forest that has been cleared". Comparing it with every object by **cosine similarity** ranks heavily deforested patches highest. That's the "semantic" score (weight α) in hybrid ranking, which is why Kisangani came out top in Q17 (0.9484).

```
cosine(a, b) = (a · b) ÷ (|a| × |b|)   → 1 = same direction, 0 = unrelated
```

In the app, a new question's vector is the **average (centroid)** of example embeddings for the chosen concept (e.g. the "current" objects of DEFORESTATION changes). The same idea, learned from examples instead of hand-built.

IF ASKED "IS THIS REAL AI?" Be honest: "The embeddings are synthetic but structured like real ones. In production, a vision model (e.g. a CNN or a remote-sensing foundation model) would produce them, and our database design and ranking would work unchanged."

Part 9: DA2 Viva Questions

Q1. Why does schema.sql start by dropping tables?

To make it safely re-runnable. CASCADE CONSTRAINTS removes FKs pointing at a table so it can be dropped in any order.

Q2. Why START WITH 10000 for the identity columns?

Sample data uses its own IDs below 10000; new rows from procedures or the app get 10000+ so they never collide.

Q3. Why index foreign keys?

Faster joins, and it avoids locking the whole child table when a parent row is deleted or updated.

Q4. What does MERGE do in compute_changes?

An "upsert": update if the (prev, curr) link exists, otherwise insert. Re-running creates no duplicates.

Q5. Why 4 alerts but 10 deforestation events?

Alerts also require confidence ≥ 0.7 and cloud ≤ 40% on both images. Q9 shows the skipped locations (Manaus, Bhadrachalam).

Q6. Why does Q4 show only 4 satellites?

Inner join; Sentinel-1A has no images. E3's LEFT JOIN shows it with 0.

Q7. Correlated vs non-correlated subquery?

Q7's subquery runs once (overall average). Q8's refers to the outer row's location and runs per row.

Q8. Window function vs GROUP BY?

GROUP BY collapses rows into one per group; a window function computes across the group but keeps every row (Q12).

Q9. How does CONNECT BY work in Q13?

START WITH picks the first patch; CONNECT BY PRIOR curr = prev follows links recursively; LEVEL is the depth.

Q10. Why use a trigger instead of a CHECK to block future dates?

A CHECK constraint can't use SYSDATE (it must be true forever); a trigger checks at insert time.

Q11. Row-level vs statement-level trigger?

Row-level (FOR EACH ROW) fires per affected row and can use :OLD/:NEW; statement-level fires once per statement (E7: 3 rows → 1 audit row).

Q12. What's an OUT parameter?

A way for a procedure to return values to the caller: add_detection returns the new id; proc_satellite_stats returns 3 numbers.

Q13. What is RAISE_APPLICATION_ERROR?

It raises a custom error with a code from −20000 to −20999 and a message, e.g. ORA-20020 Unknown object type.

Q14. What do ORA-02290 / 02291 / 02292 mean?

CHECK violated / FK parent missing / FK child records exist.

Q15. How do you know the results are correct?

The Python generator computed the expected values independently; Oracle's output matched exactly (36 links, 4 alerts, confidences to 4 dp).

Q16. What is a savepoint?

A marker inside a transaction; ROLLBACK TO it undoes only the work after it (Q19, E8, add_detection).

Q17. Why is attributes_json a CLOB with IS JSON?

Different object types have different attributes; JSON stores them flexibly, IS JSON guarantees valid JSON, and JSON_VALUE queries inside it (Q15).

Q18. What is a package's spec vs body?

Spec = the public declarations (what's callable); body = the implementation. Callers depend only on the spec.